



Факультет ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине

«Высокопроизводительные системы»

Выполнили:

Студенты группы Р3432

Носов Георгий

Карандашева Анастасия

Ястребов-Амирханов Алекси

Проверил:

Исаев Илья

Владимирович

Содержание

Содержание.....	2
1. Задание	2
2. Архитектура системы и структура проекта	4
3. Модель данных и миграции базы данных	5
4. Проектирование и реализация REST API	6
5. Реализация сложных транзакционных сценариев	8
5.1. Транзакционный сценарий №1: Назначение сотрудника на смену.....	8
5.2. Транзакционный сценарий №2: Обработка заявки на отпуск	9
6. Валидация данных, обработка исключений и контейнеризация.....	10
7. Тестирование программного обеспечения.....	11
8. Выводы	12

1. Задание

Целью лабораторной работы является написание монолита с использованием Spring Boot.

Необходимо:

- Следовать принципам разработки ПО, таким к SOLID, DRY, KISS и др.
- При разработке использовать git feature branching strategy.
- Код должен быть покрыт модульными и интеграционными тестами. Минимальный общий процент покрытия кода тестами 70%.
- Всё должно запускаться через docker compose.

Основные требования:

- Основные сущности должны иметь осмысленный CRUD интерфейс с использованием REST API.
- Использовать правильные http статусы для ответов.
- Для запросов к БД использовать Spring Data JPA/JDBC.

- Должна быть реализована валидация полей на уровне контроллера и на уровне Entity.
- Структура БД должна создаваться через Liquibase/Flyway миграции.
- Для проверки бизнес-логики должны быть написаны интеграционные тесты с использованием testcontainers и junit-jupiter-api.
- Использовать переменные среды для конфигурации приложения. При развертывании их можно задавать через поле enviroments: в docker-compose.yml.
- Приложение должно собираться с использованием docker и запускаться через docker-compose.
- Использовать любую реляционную/нереляционную БД (зависит от информационной системы), запущенную через docker.
- Каждый findAll должен иметь пагинацию. Нельзя отдавать больше 50 записей за один запрос.
- Должен быть минимум один запрос, который вернет findAll в виде бесконечной прокрутки без указания общего количества записей.
- Должен быть минимум один запрос, который вернет findAll с пагинацией и с указанием общего количества записей в http хедере.
- На сложных запросах должны использоваться транзакции. Должно быть не меньше двух подобных запросов. Обосновать почему там нужны транзакции.
- Разделять модели Entity и Dto.
- Приложение должно иметь чистый код и архитектуру с разделением по сервисам, репозиториям, контроллерам, моделям и т.д.
- Все enums в БД должны сериализоваться как строки.
- Ошибки (Exception) из контроллеров нужно хендлить и отдавать человеко читаемую ошибку в теле ответа.
- Продумать и согласовать архитектуру БД с преподавателем.
- Должны быть реализованы связи между сущностями каждого типа: Many to Many, One to Many/Many to One, Many to Many с дополнительным полем.
- Добавить интерактивную документацию с помощью OpenApi 3, развернуть Swagger (он должен быть общий) и дальше его поддерживать.

2. Архитектура системы и структура проекта

В основу архитектуры сервиса заложен принцип Package-by-layer, который даёт нам высокую модульность, тестируемость и разделение зон ответственности между программными компонентами. Структура проекта полностью изолирует внутренние детали логики от внешнего транспортного уровня.

Ключевой предметной концепцией/идеей системы мы сделали разделение понятий Планового графика смен и Фактического табеля рабочего времени:

- **Плановый график** (Schedule / Shifts): определяет целевой график работы подразделения. Менеджер формирует расписание (Schedule) на период и создает плановые смены (Shift), на которые назначаются доступные сотрудники с учетом их квалификации.
- **Фактический табель** (AttendanceRecord): фиксирует реальное присутствие сотрудника на рабочем месте. При выходе на смену работник выполняет Check-In, а при завершении — Check-Out. Система сопоставляет плановый интервал с фактическим и автоматически рассчитывает показатели дисциплины (lateMinutes — опоздание, overtimeMinutes — переработка).

Дерево пакетов проекта организовано следующим образом:

- portal.config - конфигурация Spring бинов (настройка OpenAPI 3 Swagger и Jackson ObjectMapper с модулем JavaTimeModule для корректной сериализации дат);
- portal.controller - REST-контроллеры (10 классов), обрабатывающие HTTP-запросы, выполняющие валидацию входных DTO, через @Valid и формирующие корректные HTTP-ответы;
- portal.service - сервисный слой бизнес-логики, содержащий валидацию бизнес-правил, алгоритмы расчетов и управление транзакциями (@Transactional);
- portal.repository - интерфейсы Spring Data JPA, инкапсулирующие доступ к БД и кастомные JPQL-запросы (включая поиск пересечений интервалов времени);
- portal.entity - JPA-модели, отображаемые на таблицы реляционной базы данных PostgreSQL;
- portal.dto - объекты передачи данных (Data Transfer Objects) между клиентом и сервером, разделённые на Request и Response. Они скрывают внутреннее состояние БД, исключают рекурсивные циклические ссылки при сериализации и содержат правила валидации;
- portal.exception - классы предметных исключений и единый контроллер обработки ошибок GlobalExceptionHandler.

В итоге запрос проходит через несколько уровней: сначала его принимает контроллер, затем сервис обрабатывает бизнес-логику, после чего репозиторий взаимодействует с базой данных. Результат проходит тот же путь в обратную сторону и возвращается клиенту уже в виде DTO и HTTP-ответа.

3. Модель данных и миграции базы данных

Вся схема базы данных развёртывается инструментом для контроля версий Flyway (скрипт V1_init_schema.sql).

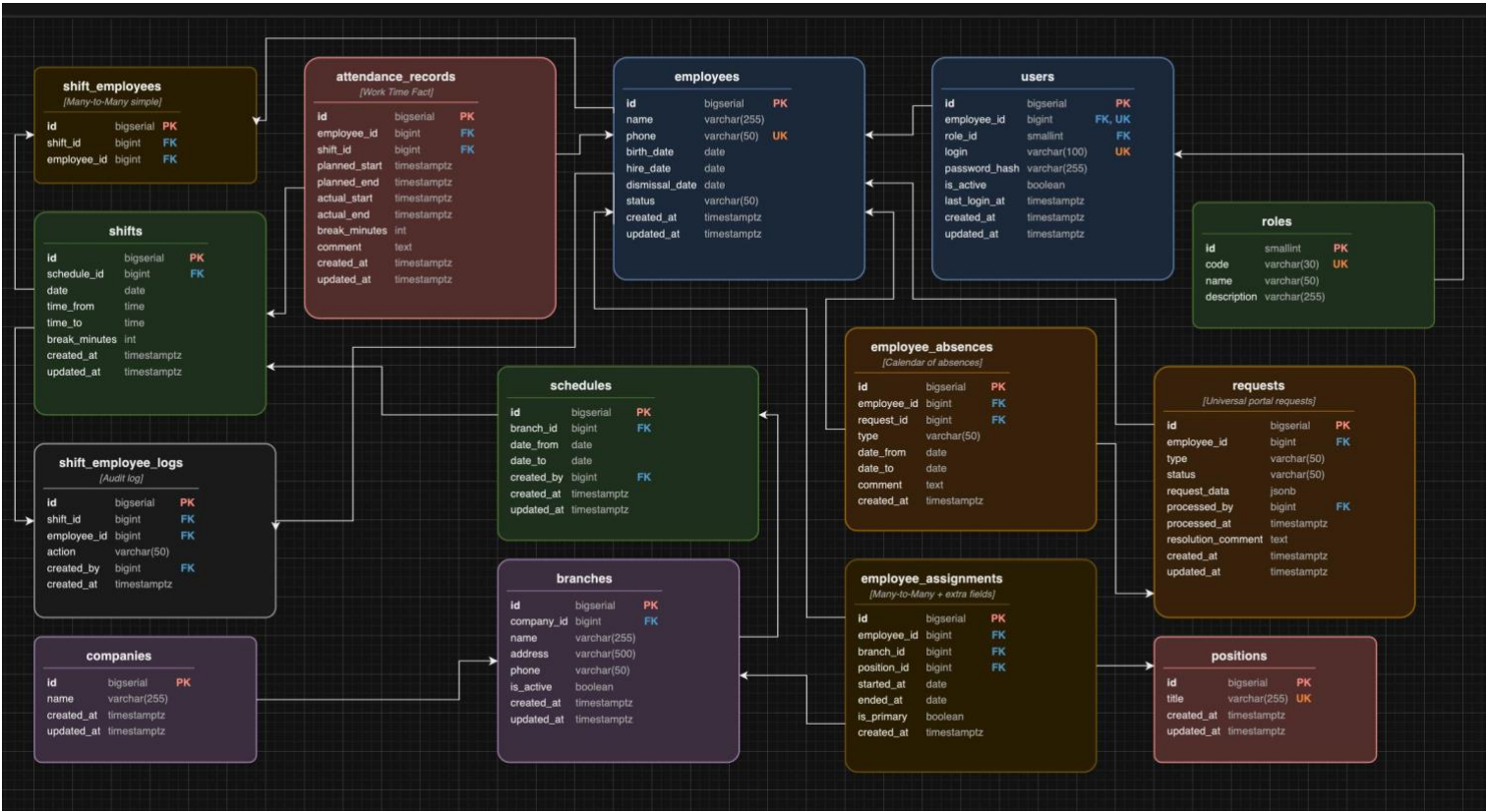


Рисунок 1. Даталогическая схема базы данных (ERD)

Таблица	Первичный ключ	Связи и ограничения	Назначение сущности
roles	id (SMALLINT)	PK: 0..3	Справочник ролей: ADMIN (0), HR (1), MANAGER (2), EMPLOYEE (3)
users	id (BIGSERIAL)	FK: role_id, employee_id (UQ), login (UQ)	Учетные записи пользователей системы
companies	id (BIGSERIAL)	1:N к branches	Юридические лица и сети заведений
branches	id (BIGSERIAL)	FK: company_id	Филиалы, офисы и торговые точки сети
positions	id (BIGSERIAL)	1:N к employee_assignments	Справочник должностей (бариста, менеджер и др.)

employees	id (BIGSERIAL)	status: ACTIVE/SICK/VACATION/DISMISSED	Личные карточки работников
employee_assignments	id (BIGSERIAL)	FK: employee_id, branch_id, position_id	История назначений сотрудника на точку и должность (М:N+)
schedules	id (BIGSERIAL)	FK: branch_id, created_by	Графики смен филиала на календарный период
shifts	id (BIGSERIAL)	FK: schedule_id	Плановые смены филиала с временными интервалами
shift_employees	id (BIGSERIAL)	FK: shift_id, employee_id (UQ пара)	Назначение сотрудников на смену (чистая Many-to-Many)
shift_employee_logs	id (BIGSERIAL)	FK: shift_id, employee_id, created_by	Неизменяемый аудит-лог изменений состава смен
attendance_records	id (BIGSERIAL)	FK: employee_id, shift_id	Отметки явок: план и фактическое время (Check- In/Out)
requests	id (BIGSERIAL)	FK: employee_id, processed_by	Заявки на отпуск, отгулы, справки с JSONB параметрами
employee_absences	id (BIGSERIAL)	FK: employee_id, request_id	Календарь зафиксированных отсутствий и отпусков

Таблица 2. Структура сущностей базы данных и схемы связей

4. Проектирование и реализация REST API

В приложении реализован REST API, через который клиент взаимодействует со всеми основными функциями системы. API разделён между 10 контроллерами, каждый из которых отвечает за свою группу операций.

При создании нового объекта сервер возвращает статус 201 Created и созданный объект в теле ответа. При успешном изменении используется 200 OK, а при удалении — 204 No Content. Если запрошенный объект не существует, сервер возвращает 404 Not Found. В случаях, когда операция нарушает бизнес-правила системы, возвращается 409 Conflict.

В соответствии с заданием в API внедрены специализированные режимы пагинации:

- **Ограничение объема выборки:** во всех эндпоинтах постраничного чтения (GET) установлен жесткий лимит size через валидацию @Min(1) @Max(50). Попытка запросить более 50 записей приводит к ошибке 400 Bad Request;
- **Бесконечная лента (Slice):** В эндпоинте GET /api/attendance реализована бесконечная прокрутка на основе org.springframework.data.domain.Slice. Вместо выполнения дорогостоящего подзапроса SELECT count(*), база запрашивает N+1 записей, определяя наличие следующей страницы (hasNext). Результат оборачивается в компактный DTO SliceResponse;
- **Метаданные в заголовках:** В эндпоинте GET /api/shifts общее число смен и количество страниц передаются в стандартизированных HTTP-заголовках X-Total-Count и X-Total-Pages, в то время как тело ответа содержит чистый JSON-массив смен.

В Таблице 3 представлена сводка основных REST-эндпоинтов системы.

Метод	Эндпоинт	Контроллер	Описание и назначение
GET	/api/companies	CompanyController	Получение списка всех зарегистрированных сетей/компаний
POST	/api/companies	CompanyController	Регистрация новой компании (201 Created)
GET	/api/branches	BranchController	Постраничный список филиалов (пагинация до 50 записей)
POST	/api/branches	BranchController	Создание филиала с привязкой к компании
GET	/api/employees	EmployeeController	Каталог сотрудников с пагинацией и сортировкой
POST	/api/employees/assignments	EmployeeController	Назначение сотрудника в филиал на должность (M:N)
POST	/api/employees/{id}/dismiss	EmployeeController	Перевод сотрудника в статус DISMISSED (увольнение)
GET	/api/schedules/branch/{id}	ScheduleController	Графики смен по конкретному филиалу
GET	/api/shifts	ShiftController	Смены с метаданными в хедерах X-Total-Count и X-Total-Pages
POST	/api/shifts/{id}/employees	ShiftController	Транзакционный сценарий №1: назначение на смену

GET	/api/attendance	AttendanceController	Бесконечная лента явок (Slice) без SELECT count(*)
POST	/api/attendance/check-in	AttendanceController	Фиксация выхода сотрудника на смену
POST	/api/attendance/{id}/check-out	AttendanceController	Закрытие смены с расчетом опозданий и переработок
POST	/api/requests	RequestController	Подача заявки на отпуск/отгул с параметрами в JSONB
POST	/api/requests/{id}/process	RequestController	Транзакционный сценарий №2: утверждение отпуска HR-ом
GET	/api/absences	AbsenceController	Календарь зафиксированных отпусков и больничных

Таблица 3. Спецификация ключевых REST-эндпоинтов системы

Для интерактивного взаимодействия и тестирования развернута документация OpenAPI 3 / Swagger UI, доступная при запуске по адресу <http://localhost:8080/swagger-ui.html>. Все схемы запросов и ответов явно снабжены аннотациями @Schema с универсальными примерами, исключающими коллизии имён классов.

5. Реализация сложных транзакционных сценариев

В сервисе выделены и реализованы два критических сценария бизнес-логики, для которых надо было обеспечить свойства ACID (атомарность, согласованность, изолированность, надежность). Управление транзакциями реализовано с помощью аннотации @Transactional от Spring Framework.

5.1. Транзакционный сценарий №1: Назначение сотрудника на смену

Зачем здесь нужна транзакция

Назначение сотрудника на смену состоит сразу из нескольких операций. Сначала проверить, может ли сотрудник выйти на эту смену, а уже после успешного назначения сохранить информацию об операции в журнале.

Все эти действия должны выполняться как одно целое. Например, ситуация, когда сотрудник уже добавлен в смену, но запись в shift_employee_logs не сохранилась,

является недопустимой. Также необходимо гарантировать, что сотрудник не окажется одновременно назначен на две пересекающиеся смены.

Последовательность операций внутри единой транзакции:

1. Проверка статуса работника: исключение назначения сотрудников со статусом DISMISSED (уволен);
2. Проверка календаря отсутствий: запрос к таблице employee_absences. Если на дату смены у сотрудника зафиксирован отпуск или больничный, выбрасывается BusinessException;
3. Проверка временных пересечений: кастомный JPQL-запрос findOverlappingShiftsForEmployee выявляет смены, на которые сотрудник уже назначен в тот же день и интервалы времени которых накладываются друг на друга;
4. Включение сотрудника в коллекцию смены (shift.getEmployees().add(employee));
5. Создание и обязательное сохранение записи в аудит-лог shift_employee_logs с фиксацией лица, выполнившего операцию.

5.2. Транзакционный сценарий №2: Обработка заявки на отпуск

Зачем здесь нужна транзакция

Одобрение заявки на отпуск затрагивает сразу несколько частей нашей системы. Нужно изменить статус самой заявки, потом добавить период отсутствия сотрудника, потом убрать его с уже назначенных на этот период смен.

Эти изменения также должны выполняться атомарно. То есть, нельзя допустить ситуацию, когда заявка уже одобрена, но запись об отпуске не создана или сотрудник продолжает числиться на сменах во время отпуска.

Последовательность операций внутри единой транзакции:

1. Проверка текущего статуса заявки: повторная обработка решенных заявок запрещена;
2. Обновление полей заявки: статус APPROVED, идентификатор согласующего лица (processedBy), дата решения и комментарий;
3. Десериализация дат начала и конца отпуска из поля JSONB requestData с помощью Jackson ObjectMapper;
4. Автоматическая вставка записи в таблицу employee_absences;
5. Поиск всех смен сотрудника в диапазоне отпуска через findShiftsForEmployeeBetweenDates, автоматическое удаление сотрудника из каждой смены и сохранение соответствующей записи в аудит-лог с действием REMOVED.

В результате обработка заявки проходит как одна транзакция: если ошибка возникает на любом из этапов, все уже выполненные изменения откатываются через rollback, и система остаётся в согласованном состоянии.

6. Валидация данных, обработка исключений и контейнеризация

В проекте используется двухуровневая валидация данных. Проверки выполняются как на уровне REST API, так и непосредственно в бизнес-логике.

На **уровне контроллеров** проверяются DTO с помощью аннотаций `jakarta.validation`, например `@NotBlank`, `@Size`, `@Pattern`, `@NotNull`, `@Min` и `@Max`. Это позволяет отсеивать некорректные запросы ещё до передачи данных в сервисный слой, что, объективно, круто.

На **уровне сервисов** выполняются уже бизнес-проверки, связанные с правилами предметной области. Например, проверяется корректность временных интервалов смен, допустимость указанных дат отпуска и уникальность логина пользователя, что, опять же, не менее круто.

Централизованная обработка ошибок реализована классом `GlobalExceptionHandler` с аннотацией `@RestControllerAdvice`. Любая ошибка перехватывается и трансформируется в стандартизированный ответ `ErrorResponse`.

Ответ содержит следующие поля:

- `timestamp` — время возникновения ошибки в формате UTC (Instant);
- `status` — HTTP-код ошибки;
- `error` — название HTTP-статуса (Bad Request, Not Found, Conflict, Internal Server Error);
- `message` — описание причины ошибки;
- `path` — URL запроса, при обработке которого произошла ошибка;
- `validationErrors` — `Map<String, String>` с указанием конкретных полей, в которых обнаружены ошибки валидации.

Такой подход позволяет возвращать ошибки в одном формате для всех REST-эндпоинтов и не дублировать обработку исключений в каждом контроллере.

Контейнеризация сервиса выполнена на двух уровнях:

- **Dockerfile:** Мультистейдж Dockerfile: на первом этапе проект компилируется и собирается в JAR-архив. На втором этапе используется более легковесный JRE-образ.
- **Docker Compose:** Файл `compose.yaml` описывает единый стек: СУБД PostgreSQL 16 (с монтированием тома `postgres_data` для сохранности данных и проверкой готовности `healthcheck` через `pg_isready`) и контейнер бэкенда, ожидающий полной готовности БД перед запуском.

Управление жизненным циклом контейнеров осуществляется командами:

- `docker compose up --build` — сборка и запуск системы в контейнерах;

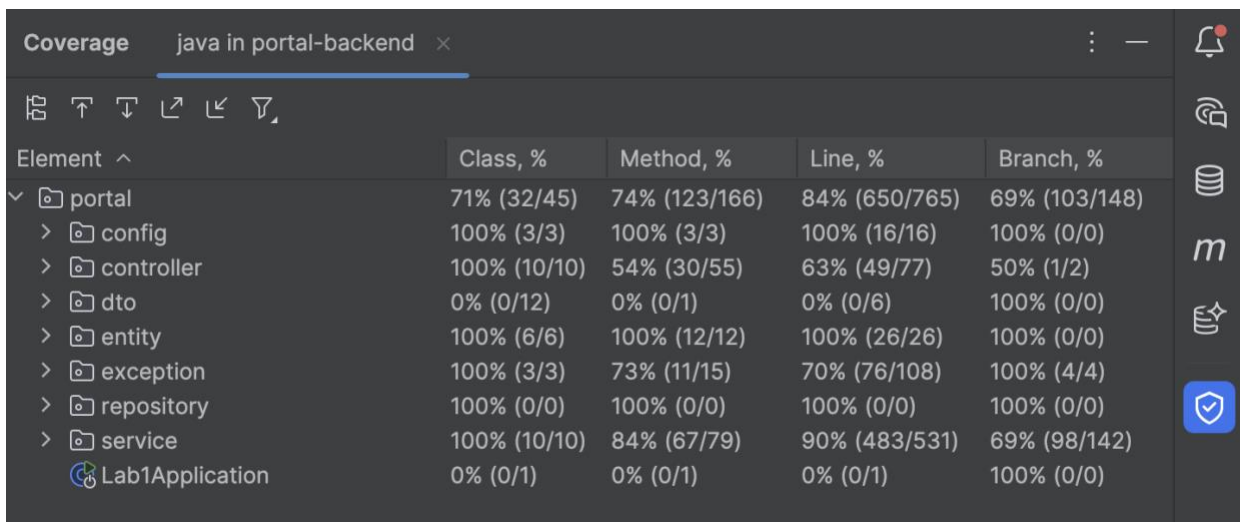
- `docker compose stop` — временная приостановка работы контейнеров;
- `docker compose down` — штатная остановка и удаление контейнеров (с сохранением базы данных);
- `docker compose down -v` — полная остановка с очисткой томов данных (полный сброс БД). //нам это не нужно

7. Тестирование программного обеспечения

Для проверки качества проекта используются два основных вида тестирования: модульное и интеграционное.

- Модульные тесты (Unit Tests): написаны с использованием JUnit 5 (Jupiter) и Mockito. Тестируют бизнес-логику изолированно от базы данных с подменой репозитория фиктивными объектами (`@Mock`, `@InjectMocks`). Это позволяет тестировать различные сценарии работы сервисов без необходимости запускать PostgreSQL.
- Интеграционные тесты (Integration Tests) реализованы с использованием библиотеки Testcontainers. В классе `Lab1ApplicationTests` во время выполнения теста автоматически запускается реальный Docker-контейнер с PostgreSQL. После этого выполняются миграции Flyway и проверяется, что приложение Spring Boot корректно запускает и загружает весь `ApplicationContext`.

Такое разделение позволяет отдельно проверять бизнес-логику и взаимодействие основных компонентов приложения с реальной инфраструктурой.



Element ^	Class, %	Method, %	Line, %	Branch, %
portal	71% (32/45)	74% (123/166)	84% (650/765)	69% (103/148)
config	100% (3/3)	100% (3/3)	100% (16/16)	100% (0/0)
controller	100% (10/10)	54% (30/55)	63% (49/77)	50% (1/2)
dto	0% (0/12)	0% (0/1)	0% (0/6)	100% (0/0)
entity	100% (6/6)	100% (12/12)	100% (26/26)	100% (0/0)
exception	100% (3/3)	73% (11/15)	70% (76/108)	100% (4/4)
repository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
service	100% (10/10)	84% (67/79)	90% (483/531)	69% (98/142)
Lab1Application	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

Рисунок 2. Тестовое покрытие

```
2026-09-25T01:39:47:487+03:00 WARN 90771 --- [portal-backend] [main] p.exception.GlobalExceptionHandler : Business
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.028 s -- in portal.exception.GlobalExceptionHandlerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 103, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 57.792 s
[INFO] Finished at: 2026-09-25T01:39:48+03:00
[INFO] -----
```

Рисунок 3. Результат прохождения тестов

Все разработанные модульные и интеграционные тесты успешно проходят без ошибок, что подтверждает корректность реализованной функциональности и стабильность приложения

8. Выводы

В ходе лабораторной работы было разработано монолитное веб-приложение на базе Spring Boot для управления сотрудниками, филиалами, сменами, рабочим временем и заявками.

Для взаимодействия с приложением реализован REST API с корректными HTTP-статусами, валидацией входных данных и пагинацией. Для хранения данных используется PostgreSQL, а структура базы управляется с помощью Flyway. Реализованы транзакционные сценарии назначения сотрудников на смены и обработки заявок на отпуск, в которых несколько связанных операций выполняются атомарно. Для защиты целостности данных используются проверки пересечений смен, периодов отсутствия и других бизнес-ограничений.

Для проверки работоспособности проекта были разработаны модульные и интеграционные тесты с использованием JUnit 5, Mockito и Testcontainers. Все разработанные тесты проходят успешно без ошибок, что подтверждает корректность основных сценариев и стабильность реализованной системы.

Приложение запускается в контейнерах с помощью Docker и Docker Compose. В состав стека входят backend и PostgreSQL, база данных сохраняется в отдельном Docker-томе, а запуск приложения выполняется только после проверки готовности БД.